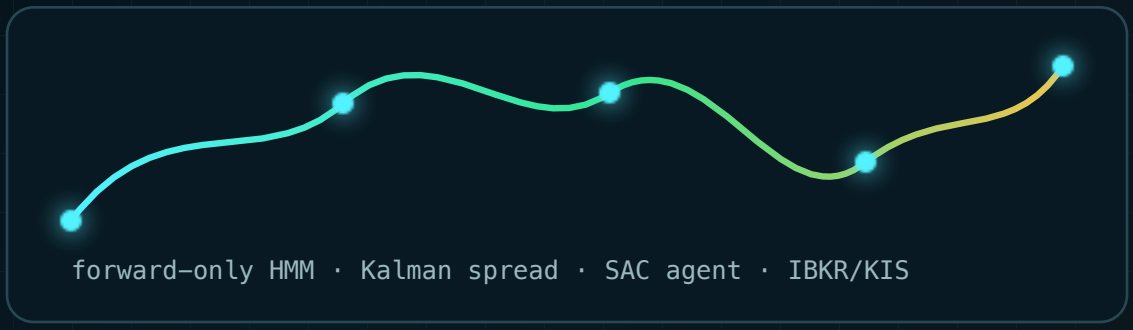
AI QUANT TRADING WITH PYTHON

Pythonで実装する AIクオンツ トレーディング

HMM·カルマンフィルタ·統計的裁定取引から

強化学習·バックテスト·リスク管理まで



Ted Park

依頼 | 実装 | 検証 | 反復

Pythonで実装するAIクオンツトレーディング

HMM・カルマンフィルタ・統計的裁定取引から強化学習・バックテスト・リスク管理まで

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Hiragino Kaku Gothic ProN | Menlo.

目次

まえがき **1**

この章の流れ 1
学習目標 2
チャートに騙された日 2
テクニカル分析の現実 2
ルネサンス・テクノロジーズが見抜いたもの 3
市場を見るもう一つの方法 3
なぜ本書を書いたか 4
AI コーディングアシスタントが可能にしたこと 5
本書を機械学習トレーディングの学習書として読む 5
迷ったときに戻る地図 7
1日を追う全体例 8
モデルをもう少し分解して考える 8
本書が扱うものと扱わないもの 9
本書を読み終えたときに作れるもの 10
ルネサンスの事例から取り出した原則 11
1. 因果性の原則 11
2. アンサンブルの原則 11
3. ウォークフォワード検証の原則 11
これからの道のり 12
参考:バックテストの数字をどう読むか 12
全章ロードマップ 13
回ごとに読み方を変える 14
Partごとの最低理解目標 15

AI をペアプログラマとして使う 16
この章で必ず覚えておくこと 16
前提知識 16

第0章 バイブコーディング・クイックスタート：まず小さな例を検証する 18

最初に期待する結果を決める 19
本文だけで実行できる確認コード 19
AIへの依頼方法 20
時間の順序を確認する 20
失敗の原因を絞り込む 21
次へ進む前の確認 21

まえがき

本書では、別途のソースファイル、サンプルZIP、リポジトリへのアクセス権は提供しない。各章の独立実行例は、本文のコードを手元のファイルに保存して実行する。`app.trading`などを含むその他の抜粋はシステム構成を説明する例であり、そのまま実行できる完全なアプリケーションではない。

この章の流れ

このまえがきは、本書の方向性を最初に固めるためのものだ。チャート中心の判断がなぜ限界にぶつかるのかを見たうえで、本書がどのような方法でクオンツシステムを組み立てるのかを確認する。

- チャートに騙された経験
- テクニカル分析が今の市場で揺らいでいる理由
- ルネサンス・テクノロジーズの事例から得られる中心的な視点
- 本書が扱うものと扱わないもの
- これから作るシステムと必要な準備

学習目標

このまえがきを読み終えたとき、次のことを説明できるようになっているはずだ。

- なぜ本書はチャートパターンではなく、データパイプラインと検証から始めるのか
- 「未来のデータをこっそり参照してしまう誤り (Look-ahead Bias)」が、なぜクオンツシステムにとって致命的なのか
- HMM、強化学習、統計的裁定取引が、本書のなかでそれぞれどんな役割を担うのか
- 学習、推論、特徴量、欠損値といった機械学習の基本用語が、トレーディングシステムの中で何を意味するのか
- 本書を読み始める前に、どの程度の Python と統計の知識が必要なのか

チャートに騙された日

ローソク足チャートを長いあいだ見つめて、買い時だと判断したことがある。同じ支持線で 3 度も反発しており、RSI は売られすぎ圏にあり、MACD のゴールデンクロスも教科書通りの形をしていた。複数のシグナルが同じ方向を指しているように見え、私はそのシグナルを信じて買いを入れた。

翌日、価格はその「完璧な」支持線をそのまま下抜けていった。初めて経験することでもなく、最後でもなかった。ただ、その日から私の問いは少し変わった。単に自分のチャートの読み方が下手だったのか、それともそもそも自分が頼っていた道具が、今の市場を説明するには不十分なのか。それを真剣に確かめる必要があった。

テクニカル分析の現実

それから数ヶ月のあいだ、論文、トレーディングフォーラム、成績データを調べながら同じ問いを繰り返した。結論から言えば、自分が頼りにしていた道具は今の市場を説明するには足りなかった。

伝統的なテクニカル分析とは、ローソク足パターン、移動平均のクロス、ボリンジャーバンド、RSI といった道具のことを指す。これらは市場の動きがゆっくりで、人間の心

理が価格を強く揺さぶっていた時代に作られた。当時はたしかに機能していた。今は違う。

現在の市場では、アルゴリズムが人間よりもはるかに速く注文をさばき、取引量の大半をアルゴリズムが占めている。かつては群衆心理を反映していたパターンが、今では機械が先に検知し、先に取引する合図になってしまった。

チャート上に支持線が見えるのは人間だけではない。アルゴリズムも見ている。ストップロス注文が集まっている地点は流動性が積み上がっている地点であり、そこは意図的に狙われることが少なくない。古い市場でよく機能していた合図が、今の市場では同じようには働かないかもしれない、ということだ。

悲観的な話をしたいわけではない。個人投資家がよく使う道具の多くは、今とは違う市場を前提に作られたという事実を指摘しているだけだ。

だから本書は「もっとうまくチャートを読む方法」から始めない。代わりに、市場をデータを生み出す一つのプロセスとして見る。そのなかに潜む状態を推定し、その推定が未来のデータをこっそり参照していないかを検証する方向へ進めていく。

ルネサンス・テクノロジーズが見抜いたもの

このアプローチを最も早く、そして最も徹底して実践したのがルネサンス・テクノロジーズだ。

ジム・シモンズは1978年にMonemetricsを設立し、1982年にRenaissance Technologiesへ改称しました。Medallion Fundの運用開始は1988年です。同社は株価チャートを読むトレーダーだけに頼るのではなく、数学者、物理学者、コンピュータ科学者を採用し、価格データに隠れた統計的パターンを探しました。公に報じられた成績に基づけば、Medallionは非常に優れた長期運用実績で知られています。

市場を見るもう一つの方法

サイモンズが生まれつき特別な直観を備えていたわけではない。違いは、市場を見る方法にあった。

ルネサンスは、市場が一つの固定された法則で動いているとは考えなかった。代わりに、市場はいくつかの「隠れた状態(hidden state)」のあいだを行き来していると見た。隠れた状態とは、表には現れないが市場を支配している局面のことだ。ある時期は強いトレンドが続き、ある時期は上がった価格が平均に戻ろうとし、ある時期は急にボラティリティが膨らむ。

これらの状態を直接目で見ることはできない。だがデータから推定することはできる。今の市場がどの状態にあるのかを把握できれば、その状態に合った戦略を選べる。

HMM(隠れマルコフモデル、Hidden Markov Model)はまさにこの「見えない状態をデータから推定する」問題を扱う道具だ。本書の中盤で詳しく取り上げる。

> 私はルネサンス・テクノロジーズの内部関係者ではなく、その会社の正確な手法を知っているわけでもない。ただ公開されているインタビューや学術研究をもとに、その思想の一部をコードとして実装してみることはできた。本書はその過程の記録である。

なぜ本書を書いたか

クオンツトレーディングを学ぶのは難しい。一つひとつの概念が難しいからではなく、同時に揃えなければならないものが多すぎるからだ。統計、信号処理、Python の実装、バックテストエンジン、ブローカー連携、リスク管理。そのすべてが必要になる。

その中で最も厄介なのが、**システムが自分自身を騙さないように作ること**だ。

「Look-ahead Bias」という落とし穴がある。戦略を組むときに、未来のデータをこっそり参照してしまう誤りだ。コードのあちこちに潜んでいて、見つけるのが容易ではない。検証をきちんとしないと、未来を覗き見した戦略が、まるで素晴らしい成績を上げたかのように見えてしまう。市場平均を超える超過収益はふつう「アルファ(alpha)」と呼ばれるが、それが本当はアルファではなく、未来の情報を写しただけの結果ということもあり得る。

クオンツトレーディングを始めようとする人の大半は、この準備を整えられないまま諦める。そしてまたローソク足と RSI に戻っていく。それで十分だからではなく、手近で扱いやすいからだ。私はこのギャップを埋めたかった。複雑な数学と実装を隠さず、しかし読者がついていける順番でほどいていきたかった。

AI コーディングアシスタントが可能にしたこと

2023 年以降、AI コーディングアシスタントは実用的な水準に達した。そのおかげで本書を書くことができた。GitHub Copilot、Claude、ChatGPT のような道具は、複雑なアルゴリズムを説明し、見つけにくい実装ミスを拾い上げ、数式がコードのなかでどんな意味を持つのかを解きほぐすのに役立つ。理解そのものを代わりにやってくれるわけではない。だが、理解にたどり着くまでの時間と試行錯誤は大きく減らしてくれる。

私は本書のシステムを AI と一緒に組み上げた。難しい数式に詰まったとき、見つけにくいバグを追ったとき、ハイパーパラメータ(モデルの性能を左右する設定値)を決めるとき、いずれも AI の力を借りた。本書はその過程をもう一度最初からたどり直す。理論の解説で止まらず、今手に入る道具で実際のシステムをどう作るのかを見せることが目標だ。

本書を機械学習トレーディングの学習書として読む

本書はHMMと強化学習を扱うが、より大きく見れば、機械学習と深層学習で株式トレーディングシステムを作るための学習書である。

基本の流れは単純だ。

```
株式市場
  -> データ
  -> 特徴量
  -> 機械学習モデル
  -> 深層学習 / RLエージェント
  -> 検証
  -> 注文実行
```

モデルは明日の価格を当てる魔法の箱ではない。過去データから内部のパラメータを学習した関数だと考えるとよい。普通の関数は人がルールを直接書く。機械学習モデルも入力と出力の形は同じだが、内部の重みをデータから学習する。

> 今日の特徴量1行 –> 学習済みモデル –> レジーム確率 / 行動提案

それでも**モデル**という言葉が曖昧に感じるなら、次のように考えるとよい。

モデル = データから内部の数値を学習した判断計算機

普通の計算機は2 + 3を入れると5を返す。トレーディングモデルは、リターン、ボラティリティ、出来高、Zスコアのような数値を受け取り、「平均回帰レジーム70%」や「許容ポジションの35%だけ使う」といった判断を返す。モデルはデータそのものではない。アルゴリズム名だけでもない。アルゴリズムがデータから生成した、学習済みの判断関数である。

```
def model(today_features):
    return prediction
```

だから本書では、次の3つの言葉を分けて読む。

言葉	やさしい意味	本書での例
アルゴリズム	モデルを作る方法	HMMのBaum-Welch、SAC学習
パラメータ	モデル内部で学習される数値	HMMの遷移行列、ニューラルネットの重み
モデル	学習済みの判断関数	今日の特徴量 -> レジーム確率

料理にたとえるなら、データは材料、アルゴリズムはレシピ、パラメータは火加減と調味料の比率、モデルはその比率が決まった調理方法である。同じレシピでも、どんな材料で練習したかによって出来上がりは変わる。機械学習も同じだ。同じHMMアルゴリズムでも、どの市場期間で学習したかによって別のモデルになり得る。

学習は過去データでモデルを合わせること。推論は学習済みモデルを固定し、新しいデータを1行ずつ入れて判断を得ること。OOS検証は、後半のデータを学習時には隠しておき、学習済みモデルがそこでまだ通用するかを確かめる作業である。

最初からすべてのアルゴリズム名を理解しようとしなくてよい。まずは **データ -> 特徴量 -> 学習 -> 推論 -> 検証 -> 注文** という骨格だけをつかむ。HMM、SAC、リスク管理、MLflowはすべてこの骨格に接続される。

迷ったときに戻る地図

同じ内容をトレーディングの言葉に置き換えると次のようになる。

機械学習の言葉	本書でのトレーディング上の意味
データ	過去価格、出来高、マクロ指標、ブローカー記録
特徴量	モデルが読める数値。リターン、ボラティリティ、モメンタム、z-scoreなど
学習	過去データからモデル内部の数値を合わせること
推論	学習済みモデルに今日の新しい1行を入れて判断を得ること
OOS	学習時に隠しておく模擬試験区間
サービング	学習済みモデルをトレーディングエンジンから呼べるサービスとして動かすこと
リスクレイヤー	モデルの提案を最後に止めたり小さくしたりする安全装置

迷ったらこの表に戻ればよい。難しく見える章の多くは、この表の1行を詳しく扱っているだけである。2章はデータと特徴量、4〜5章はHMMの学習と推論、13〜14章はRLエージェント、16章はリスク、27章は学習済みモデルのサービングを扱う。

初回の通読に使うチェックリストは次のとおりである。

> [ ] モデルに入るデータを説明できるか。
> [ ] そのデータから作る特徴量を説明できるか。
> [ ] 学習データと推論データを分けられるか。
> [ ] OOS区間がどこかを指せるか。
> [ ] モデルが何を出力するかを言えるか。
> [ ] 注文前にどのリスクチェックが入るかを言えるか。

この問いに答えられれば、HMM、SAC、Kelly、CVaRがまだ難しく感じても、本書の本当の構造は追えている。パイプラインが見えれば、細部はあとから理解しやすくなる。

1日を追う全体例

具体例で見ると分かりやすい。5年分の日足データがあるとしよう。初心者向けの分け方なら、最初の2年でモデルを学習し、残り3年で推論と検証を行う。後半3年もディスク上では過去データだが、検証中は未来データのように扱わなければならない。モデルはそこに合わせ込んではいけない。1行ずつ特徴量を受け取り、出力だけを返す。

```
5年分の特徴量

最初の2年: 学習データ
  -> スケーラーを合わせる
  -> モデルを学習する
  -> モデルパラメータを保存する

次の3年: OOS推論データ
  -> 学習済みモデルをロードする
  -> 1行ずつ特徴量を入れる
  -> モデル出力を記録する
  -> リスクチェックを通す
```

モデルをもう少し分解して考える

モデルファイルが何を保存しているのかを理解するにも、この例が分かりやすい。モデルファイルは学習データそのものではない。学習の結果である。学習済みパラメータ、スケーラー、マッピング規則、場合によっては推論を続けるための最後の状態が入っている。サービングとは、そのモデルファイルを読み込み、リクエストに答えられる状態にすることだ。

```
学習がモデルファイルを作る
サービングがモデルファイルをロードする
推論が新しい特徴量をロード済みモデルへ送る
リスクがモデル出力を注文にしてよいか決める
```

同じパターンは本書全体に繰り返し現れる。HMMの学習はレジームパラメータを作る。HMMの推論は今日のレジーム確率を読む。SACの学習はActorを作る。SACの推論は行動を提案する。MLflowは、どの学習実行からどのモデルファイルが生まれたかを記

録する。リスク管理は、モデルの出力が大きすぎる注文にならないよう、最後の段階で抑えたり止めたりする役割を担う。

実運用では、この流れは2つの時間軸に分かれる。

```
遅い時間軸: 学習 / 検証 / モデル登録
過去データ -> 学習 -> OOS検証 -> モデルファイル保存 -> 承認

速い時間軸: リアルタイム推論 / 注文
今日のデータ -> 特徴量 -> モデルファイルをロード -> 推論 -> リスク確認
-> 注文 -> 約定確認 -> モニタリング
```

取引時間中のモデルは、毎分もう一度勉強している学生ではない。試験会場で答えを出す計算機に近い。学習は別の時間に終わっている。ライブシステムは承認済みのモデルファイルを読み込み、今日の特徴量1行を入れ、出力を受け取り、その出力を注文にしてよいかをリスクレイヤーで判断する。

したがって、モデルファイルを保存するとは「ディスクにファイルを1つ置く」という意味だけではない。入力形式、学習済みの数値、出力の意味を一緒に固定するという意味である。学習時の順序が`[return, volatility, momentum, volume]`だったのに、推論時に`[return, volume, volatility, momentum]`を送れば、モデルは4つの数字を受け取っていても意味を間違えて読む。後の章で特徴量順序、スケーラー、レジームラベルの対応付け、サービング、運用検証を繰り返し扱う理由はここにある。

本書が扱うものと扱わないもの

本書の範囲をまずはっきりさせておく。

本書が扱わないこと

- Python の入門書ではない。ループを書ける、ライブラリをインポートできる、という前提から始める。
- チャートトレーディングの教本ではない。支持線や MACD ダイバージェンスのようなチャートベースのテクニカル分析は扱わない。
- 一攫千金の手引きでもない。クオンツトレーディングは難しく、リスクも大きい。

本書が扱うこと

クオンツトレーディングシステムを最初から自分の手で組み立てるための実践ガイドだ。ルネサンス・テクノロジーズのようなクオンツヘッジファンドが用いるタイプのシステムを、AI の支援を受けながら段階を追って構築する。本書に載っているコードは実際に動き、結果は数値で確かめられる。

本書を読み終えたときに作れるもの

システム	一行で言うと
HMM 市場局面ディテクター	未来のデータを使わず、これまでのデータだけで市場が今どの局面(上昇・横ばい・下落)にあるのかを推定する。
複数資産を同時に見る HMM	株式や債券など複数の資産のシグナルを同時に読み、より精度の高い局面把握を行う。
2 段階のマクロ/ミクロ HMM	マクロ経済と個別資産を 2 段階で見て、市場を Risk-On(リスク選好)、Neutral(中立)、Risk-Off(リスク回避)に分類する。
強化学習トレーディングエージェント	HMM が検出した局面を入力として受け取り、ポジションを自分で決める。
統計的裁定取引システム	一緒に動く資産ペアを見つけ、その価格差をねらう。
実運用デプロイシステム	IBKR(Interactive Brokers)と KIS(韓国投資証券)のブローカーに接続し、実際の注文を出す。

それぞれの部品がなぜ必要なのか、間違って作るとどんなふうに崩れるのかを理解すること。それが本書の本当の目標だ。

ルネサンスの事例から取り出した原則

本書全体を貫く原則が三つある。ルネサンス・テクノロジーズのトレーディング手法を分析するなかで取り出したものだ。

1. 因果性の原則

> 未来のデータを現在の判断に使ってはいけない。

当たり前に聞こえるが、コードに落とし込んでいくと意外と簡単に破ってしまう。よく使うライブラリ関数のなかにも、内部で未来情報をこっそり混ぜているものがある。本書ではそうした関数の代わりに、「今までのデータしか見ない」という約束を守る方法(フォワードアルゴリズム)を自分で実装する。詳しい理由は 5 章で説明する。一度理解してしまえば、以降のコードで同じ問題がすぐ目に飛び込んでくるようになる。

2. アンサンブルの原則

> 一つのモデルに頼り切らない。

このシステムでは、日足の HMM、時間足の HMM、強化学習エージェント、統計的裁定取引のシグナルを同時に使う。各モデルの判断が食い違うときは待つ。すべて一致したときに動く。

3. ウォークフォワード検証の原則

> 戦略が本当に機能するかを確かめる方法は一つしかない。

モデルが一度も見たことのないデータを、時間の順序に沿って繰り返しテストすることだ。データを 1 回だけ訓練用とテスト用に分ける方法では足りない。ウォークフォワード検証は、本書のすべての戦略に適用する基本ラインだ。

これからの道のり

システム全体は、データ -> ノイズ除去 -> 局面検出 -> ポジション判断 -> 検証と実運用、という順に積み上げる。それぞれの段階が、一つの章のまとまりとして続いていく。

段階	やること
1	市場データを集めて保存する
2	価格に混ざったノイズを減らす
3	複数資産を同時に見る HMM で市場局面を推定する
4	マクロ/ミクロの 2 段階で局面を Risk-On / Neutral / Risk-Off に整理する
5	局面に合わせてどれだけのポジションを持つかを決める
6	統計的裁定取引で追加の収益機会を探す
7	強化学習エージェントがポジション判断を補助する
8	リスク管理、ブローカー接続、モニタリングで運用条件を整える

最初に読むときは、1〜18 章を**基本システムを一周完成させる流れ**として眺めるとよい。19〜27 章は同じ素材をもう一歩深く扱う応用編だ。ペア理論、完全な HMM パイプライン、運用検証、分布型 RL、MLflow サービングといった、実際の運用段階で改めて必要になる内容をまとめた区間になっている。

参考:バックテストの数字をどう読むか

本書にはいくつものバックテスト結果が登場するが、特定の数値を将来の成績を保証するものとしては読まないでほしい。同じ戦略でも、データ期間、取引コスト、銘柄ユニバース、再学習頻度、スリッページの仮定によって結果は大きく変わる。

最初に読むときは、数字よりも次の三つを見る。

- 未来のデータを覗いていないか

- 最大ドローダウンと取引コストも一緒に確認しているか
- OOS(サンプル外)区間でも似たパターンが保たれているか

全章ロードマップ

Part	章	テーマ
I	まえがき・1〜3 章	システム全体像、Look-ahead Bias、データパイプライン、バックテストエンジン
II	4〜5 章	HMM の基本構造と学習アルゴリズム、未来情報なしで推論する方法
III	**6〜8 章**	カルマンフィルタ(ノイズ除去)、複数資産を同時に見る HMM、2 段階のマクロ/ミクロ HMM (システムの中核)
IV	9〜10 章	ポジションサイズを決める、HMM アンサンブルで方向シグナルを作る
V	11〜12 章	一緒に動くペアを見つけ、統計的裁定取引のシグナルを作る
VI	13〜15 章	強化学習環境、SAC エージェント、ハイパーパラメータチューニング
VII	16〜18 章	リスク管理、ブローカー連携、リアルタイムモニタリング

Part	章	テーマ
VIII	19〜27 章	応用編:ペア理論、レジームベースの Z スコア、完全パイプライン、運用検証、分布型 RL、MLflow サービング

> 表に出てきた用語の多くは、初見ではどれもなじみが薄い。今この場で覚える必要はない。最初の通読では Part I〜VII を通して「データから注文・モニタリングまで」を一周する。Part VIII は二周目、あるいは実システムを磨き上げるときに必要になる応用区間だと思っておけばよい。

回ごとに読み方を変える

本書は一度ですべての数式とコードを理解するための本ではない。最初からすべてのパラメータや実装の細部を追おうとすると、全体の流れを見失いやすい。読む回ごとに目標を変えるとよい。

周	目標	読む基準
1周目	大きな流れをつかむ	データがどうモデル判断と注文へつながるかを見る
2周目	コードと概念をつなぐ	各章の入力、出力、失敗点をコードで確認する
3周目	実験と修正	パラメータを変え、OOS結果がどう動くかを見る

1周目では数式を完全に理解できなくてもよい。代わりに、各章が次のどの問いに答えているかをつかむ。

> この章はデータを準備する章か?
> 市場を読む章か?
> 機会を探す章か?

> サイズを決める章か?
> リスクを止める章か?
> 運用状態を確認する章か?

2周目ではコードに降りる。`fit()`がどこで呼ばれるか、`predict()`や`step_forward()`がどの区間でだけ実行されるか、OOS区間が学習に混ざっていないかを確認する。この段階になると、数式は暗記対象ではなく、コードがなぜその形をしているかを説明する地図になる。

3周目では実験する。ウィンドウ長、エントリー閾値、レジーム数、取引コスト、リスク上限を少しずつ変える。ただし、同じOOS結果を見ながら何度もチューニングしてはいけない。値を変えるなら、どんな仮説で変えたのかを記録し、検証区間と最終テスト区間を分ける。

Partごとの最低理解目標

最初に読むとき、各Partで最低限これだけは説明できるようになっておきたい一文は次のとおりである。

Part	最低理解目標
I	データとバックテストが嘘をつかない土台を作る
II	HMMは未来価格ではなく、市場状態を確率で読む
III	ノイズを減らし、複数市場を同時に見るほどレジーム判断は安定する
IV	シグナルは方向であり、Kellyはその方向を資金サイズに変える
V	ペアトレードは一銘柄の方向ではなく、二つの価格の関係を取引する
VI	RLはすべてのシグナルを作る魔法のAIではなく、ポジションサイズを調整する層である
VII	モデルが正しくても、リスク、注文、モニタリングが壊れれば実運用は失敗する
VIII	同じ材料をより厳しく検証し、運用可能な形へ整える

この表を最初の基準にすればよい。細かい数式がまだ難しくても、各Partの最低目標を説明できれば1周目は十分に成功である。

AI をペアプログラマとして使う

本書では、行き詰まったところごとに AI コーディングアシスタントをどう活用したかを直接見せていく。アルゴリズムを理解するために使ったプロンプト、AI が拾ってくれた見つけにくいバグ、AI のおかげでより良い方向に修正できた瞬間が、本文のあちこちに織り込まれている。

ねらいは思考そのものを AI に渡してしまうことではない。**頼れる先輩エンジニアに相談するように**使えばいい。分からないことは聞き、理解できたものは確認し、知っている部分はさっと飛ばし、詰まった部分は一緒にほどいていく。

クオンツトレーディングは博士号を持つ人だけのものではない。道具と方向さえきちんと選べば、実用的なトレーディングシステムを自分の手で組み立てられる。本書はその過程の記録だ。

この章で必ず覚えておくこと

- 本書はチャートパターンをより正確に覚えるための本ではない。市場を「データで読み解くシステム」を作るための本だ。
- 最も重要な約束はたった一つ、**「未来のデータを現在の判断に使わない」**ことだ。この約束が破れた瞬間、見栄えの良い成績はすべて偽物になる。
- 一つのモデルにすべてを賭けない。複数のモデルが同じ方向を指したときだけ動く。
- 最初からすべての数学を完璧に押さえる必要はない。各章で「なぜこの概念が必要なのか」「間違って作るとどんな問題が起きるのか」を順番に追っていけばよい。

前提知識

本格的な内容に入る前に、以下くらいは準備しておいてほしい。

- **Python 3.10 以上**
- **NumPy と Pandas の基本的な使用経験** —配列を扱い、データフレームを操作できる程度で十分だ。
- **基礎的な統計の概念** —平均、分散、正規分布が何かを知っている程度で構わない。それより難しい概念は、必要なところで本のなかで説明する。
- **市場がなぜそんなふうに動くのかを知りたいという好奇心**

準備ができたなら、次の章でシステム全体の設計図を広げていこう。

第0章 バイブコーディング・クイックスタート：まず小さな例を検証する

この章の目的は、小さな数値例を使い、**AIが説明した損益が正しいかを自分で確認すること**だ。著者の運用リポジトリ、証券口座、市場データは必要ない。Pythonを実行できるなら以下のコードを動かす。実行環境がなければ表を手計算すればよい。

最初に期待する結果を決める

現金1,000から始め、価格100の資産を10単位購入する。その後、価格は80まで下がり、100に戻る。最初の計算では費用をゼロとする。

時点	現金	保有数量	価格	総資産
取引前	1,000	0	100	1,000
購入後	0	10	100	1,000
価格下落時	0	10	80	800
価格回復時	0	10	100	1,000
決済後	1,000	0	100	1,000

総資産は現金 + 数量 × 価格だ。最終損益はゼロだが、最大ドローダウンは`(1000-800)/1000=0.2`、つまり20%になる。最終残高だけを見ると、途中で負ったリスクを見落とす。

本文だけで実行できる確認コード

次のコードを`first_check.py`という新しいファイルに貼り付ける。Pythonの標準機能だけを使うため、追加ファイルのダウンロードは不要だ。

```
prices = [100.0, 80.0, 100.0]
initial = 1000.0
quantity = initial / prices[0]
cash = initial - quantity * prices[0]
equity = [initial] + [cash + quantity * p for p in prices]
peak = initial
mdd = 0.0
for value in equity:
    peak = max(peak, value)
    mdd = max(mdd, (peak - value) / peak)
final_cash = cash + quantity * prices[-1]
```

```
assert final_cash == 1000.0
assert mdd == 0.2
print(final_cash, mdd)
```

ターミナルでpython3 first_check.pyを実行すると、1000.0 0.2と表示される。WindowsでPythonのコマンドがpyなら、py first_check.pyを使う。これは数量を固定した損益の検算であり、シグナル生成、実注文、HMMの学習は含まない。

AIへの依頼方法

> 上の表とPythonコードを一緒に確認してください。
> 1. 総資産と最大ドローダウンを手計算してください。
> 2. 現金だけを記録すると、どのリスクを見落とすか説明してください。
> 3. 購入と売却にそれぞれ10 bpの手数料がかかる場合、最終現金を計算してください。
> 4. 証券会社、外部データ、新しいパッケージを追加しないでください。
> 回答後、表のどの値で結果を検証したか示してください。

10 bpは取引金額の0.1%だ。10単位を価格100で購入・売却すると、それぞれの費用は1、合計2となり、最終現金は998になる。この例では初期現金の全額を使って数量を決めるため、購入手数料の1だけ現金が一時的にマイナスになる。借入を認めない実際の口座では、手数料分を残すよう購入数量を減らす必要がある。固定数量の演習と実口座の制約を区別しよう。

時間の順序を確認する

今日の終値を見て作ったシグナルが、それより前の今日の始値で約定したとは仮定できない。本書の基本演習は、**t本目の終値後にシグナル確定 -> t+1本目の始値で約定 -> t+1本目の終値で評価**という順序だ。前もって確定したシグナルに従い始値で建てたポジションについて、その日の損益を含めることには問題がない。

失敗の原因を絞り込む

症状	最初に確認すること
Pythonコマンドが見つからない	インストール済みのコマンドがpython3かpyかを確認する
インデントエラー	コード部分だけをコピーし、字下げを保ったか確認する
ドローダウンがゼロになる	価格80の時点で総資産800を記録したか確認する
最終現金と損益合計が一致しない	損益の二重加算や費用の重複を確認する

新しいパッケージを入れる前に、上の例と異なる行を確認する。生成された説明のもっともらしさよりも、期待値と実際の出力が一致することを優先する。

次へ進む前の確認

- 最終損益ゼロと最大ドローダウン20%が同時に成立する理由を説明できる。
- 手数料を実際の約定ごとに一度だけ計上する。
- シグナル生成時刻と約定時刻を区別する。

この三つを確認したら、第1章の構成説明へ進もう。別途のサンプルファイルは提供しないため、実習では本文のコードを保存して使う。本文の`app.trading`などの名前は、著者のシステム構成を説明する例であり、概念を理解するために非公開パッケージへアクセスする必要はない。

続きは完全版で

この試し読みはここまでです。完全版には、以下の章が含まれます。

- まえがき
- 第0章 バイブコーディング・クイックスタート：まず小さな例を検証する
- 第1章 システムアーキテクチャ — AIクオンツトレーディングの設計思想
- 第2章 データパイプライン—DuckDBとリアルタイムフィード
- 第3章. バックテストエンジン—信頼できるシミュレーション
- 第4章 HMMの基礎—市場レジームを確率で推論する
- 第5章. HMM Forward-Only —未来情報を使わないリアルタイム推論
- 第6章. カルマンフィルター—ノイズ除去とイノベーションZスコア
- 第7章 クロスアセットJoint HMM —複数市場を同時に見るレジーム検出
- 8章. 2階層HMMアンサンブル—Layer A × Layer B
- 第9章 ハーフ・ケリーによるポジションサイジング
- 第10章. HMM フォワードアンサンブル—V1 -> V3 への進化
- 第11章 統計的裁定取引—ペア選定
- 第12章 統計的裁定取引—シグナル生成
- 第13章 RL環境の設計—StatPairRLEnv
- 第14章. SACエージェント—アーキテクチャと訓練

- 15章. Optuna ハイパーパラメータチューニング
- 第16章 リスク管理—ポジション保護とリスク指標
- 第17章. IBKR接続—リアルタイム注文の約定
- 第18章. モニタリング—リアルタイム成績追跡とアラート
- 第19章. バンバーガーの統計的ペア裁定取引—理論と原則
- 第20章 ペア選定—レジーム条件付きフィルタリング
- 第21章. レジーム連動Zスコア—戦略ロジックの中核
- 第22章. HMMレジーム戦略—データからシグナルまでの完全なパイプライン
- 第23章 SAC RLエージェントプレビュー—強化学習入門
- 第24章 本番運用の検証
- 25章. RLアルゴリズム比較—SAC vs PPO vs QR-DQN
- 第26章 分布型RLとCVaR —テールリスクを意識した取引
- 第27章 MLflowモデルレジストリとサービング—本番デプロイ
- 付録. 初心者のための全体地図

完全版はこちら